HUNDRED DAYS OF CODE
DAY 35

QUESTION: Write a menu-driven C program to read a variable-length square integer array and perform the following operations using separate functions:

1. Determine whether all the elements on its main diagonal are distinct.
2. Find the sum of its main diagonal elements.
3. Display its elements using a diagonal traversal. Begin with the element in the top-left corner and traverse successive diagonals alternately in the upward-right and downward-left directions.
4. Exit.

Example: For the following input matrix

image

1. Main diagonal elements: 1 5 9. All the main diagonal elements are distinct.
2. Sum of main diagonal elements: 15
3. Diagonal traversal: 1 2 4 7 5 3 6 8 9

SOLUTION:

1. Aim

To write a menu-driven C program using **separate functions and a variable-length square 2D integer array** to:

1. Determine whether all elements on the main diagonal are distinct.
2. Find the sum of the main diagonal elements.
3. Display the matrix elements using alternate **diagonal traversal**.
4. Exit the program.

2. Algorithm

Main Algorithm

1. Start the program.
2. Input the order n of the square matrix.
3. Declare a variable-length 2D array A[n][n].

HUNDRED DAYS OF CODE
DAY 35

4. Input the matrix elements.
5. Display the menu:
    - Check whether main diagonal elements are distinct.
    - Find the sum of main diagonal elements.
    - Perform diagonal traversal.
    - Exit.
6. Accept the user's choice.
7. Use a switch statement to call the appropriate function.
8. Repeat the menu until the user selects **Exit**.
9. Stop.

Operation 1: Check Distinct Main Diagonal Elements

1. Access the main diagonal elements using A[i][i].
2. Compare each diagonal element with all diagonal elements after it.
3. If any two elements are equal, set a flag indicating that the elements are not distinct.
4. Display all main diagonal elements.
5. If no duplicate is found, display that all diagonal elements are distinct.
6. Otherwise, display that the diagonal elements are not distinct.

Operation 2: Sum of Main Diagonal Elements

1. Initialize sum = 0.
2. Traverse from i = 0 to n - 1.
3. Add A[i][i] to sum.
4. Display the final sum.

HUNDRED DAYS OF CODE
DAY 35

Operation 3: Diagonal Traversal

1. Start from the top-left element A[0][0].
2. Consider all diagonals having the same value of row + column.
3. Traverse successive diagonals in alternating directions.
4. For one diagonal, move **upward-right**.
5. For the next diagonal, move **downward-left**.
6. Continue alternating the direction until all matrix elements have been displayed.

C PROGRAM:

```
#include <stdio.h>


// Function declarations
void checkDistinct(int n, int A[n][n]);
void diagonalSum(int n, int A[n][n]);
void diagonalTraversal(int n, int A[n][n]);


int main()
{
   int n, i, j, choice;


   // Input matrix order
   do
   {
      printf("Enter the order of the square matrix: ");
      scanf("%d", &n);


      if(n <= 0)
```

HUNDRED DAYS OF CODE
DAY 35

```
        printf("Order must be positive. Please try again.\n");

} while(n <= 0);

// Variable Length Array
int A[n][n];

printf("\nEnter the matrix elements:\n");

for(i = 0; i < n; i++)
{
   for(j = 0; j < n; j++)
   {
      scanf("%d", &A[i][j]);
   }
}

do
{
   printf("\n----- MATRIX OPERATIONS MENU -----\n");
   printf("1. Check distinct main diagonal elements\n");
   printf("2. Sum of main diagonal elements\n");
   printf("3. Diagonal traversal\n");
   printf("4. Exit\n");

   printf("Enter your choice: ");
```

HUNDRED DAYS OF CODE
DAY 35

```
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            checkDistinct(n, A);
            break;

        case 2:
            diagonalSum(n, A);
            break;

        case 3:
            diagonalTraversal(n, A);
            break;

        case 4:
            printf("Program exited successfully.\n");
            break;

        default:
            printf("Invalid choice! Please try again.\n");
    }

} while(choice != 4);
```

HUNDRED DAYS OF CODE
DAY 35

```
    return 0;
}



// Function to check whether main diagonal elements are distinct
void checkDistinct(int n, int A[n][n])
{
    int i, j;
    int distinct = 1;

    printf("\nMain diagonal elements: ");

    for(i = 0; i < n; i++)
    {
        printf("%d ", A[i][i]);
    }

    // Compare diagonal elements
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(A[i][i] == A[j][j])
            {
                distinct = 0;
                break;
```

HUNDRED DAYS OF CODE
DAY 35

```
        }
      }

      if(distinct == 0)
        break;
   }

   if(distinct == 1)
      printf("\nAll the main diagonal elements are distinct.\n");
   else
      printf("\nThe main diagonal elements are not distinct.\n");
}


// Function to find sum of main diagonal elements
void diagonalSum(int n, int A[n][n])
{
   int i;
   int sum = 0;

   for(i = 0; i < n; i++)
   {
      sum = sum + A[i][i];
   }

   printf("\nSum of main diagonal elements = %d\n", sum);
```

HUNDRED DAYS OF CODE
DAY 35

```
}



// Function for diagonal traversal
void diagonalTraversal(int n, int A[n][n])
{
   int d, i, j;

   printf("\nDiagonal traversal: ");

   // d represents sum of row and column indices
   for(d = 0; d <= 2 * (n - 1); d++)
   {
      if(d % 2 == 0)
      {
         // Traverse upward-right
         i = (d < n) ? d : n - 1;
         j = d - i;

         while(i >= 0 && j < n)
         {
            printf("%d ", A[i][j]);
            i--;
            j++;
         }
      }
```

HUNDRED DAYS OF CODE
DAY 35

```
    else
    {
      // Traverse downward-left
      j = (d < n) ? d : n - 1;
      i = d - j;

      while(j >= 0 && i < n)
      {
        printf("%d ", A[i][j]);
        i++;
        j--;
      }
    }
  }

  printf("\n");
}
```

TEST CASE TABLE:

Test Case	Matrix	Operation	Expected Result
1	1 2 3 / 4 5 6 / 7 8 9	Distinct diagonal	1 5 9 → All distinct

HUNDRED DAYS OF CODE
DAY 35

Test Case	Matrix	Operation	Expected Result
2	1 2 3 / 4 1 6 / 7 8 9	Distinct diagonal	1 1 9 → Not distinct
3	1 2 3 / 4 5 6 / 7 8 9	Diagonal sum	15
4	1 2 3 / 4 5 6 / 7 8 9	Diagonal traversal	1 2 4 7 5 3 6 8 9
5	5	All operations	Distinct, Sum = 5, Traversal = 5
6	Any valid matrix	Exit	Program terminates
7	Any valid matrix	Invalid choice	Displays invalid choice message

HUNDRED DAYS OF CODE
DAY 35

COMPILATION AND EXECUTION:



